

OOP:

Drawbacks of Functional / Procedural Programming

1. Does not model real world problem very well.
2. If a new data is added, all the functions needs to be modified to access the Data.
3. Global data is accessible to all the functions.
4. No clear boundaries and well definition of code.
5. No Modularity: Functional programs can become monolithic and difficult to maintain as they grow in complexity.

What Is OOP?

Approach where problems are structured around objects rather than function and logic, these objects can communicate with each other in controlled manner.

What is OOP?

1. Programming is used to solve real-world problems, how can we model real-world systems with programming languages.
2. A Programming Style, involves dividing a program into pieces of objects that can communicate with each other.
3. Objects based coding style, in which each object (aka, real-world entity) has its own attributes and behaviour.
4. Fundamental Idea is to combine into single unit, both data and behaviour, that will promote Modularity.
5. **OOP promotes modularity** by encapsulating data and behavior within objects. This modular approach enhances code reusability and maintainability, as objects can be reused in different parts of the program.
6. OOP is LIFE (We'll understand this on-the-go)

Each class (real world entity set) has its own attributes (properties) and behaviours (functions/ methods).

Each object (instance of entity set - entity).

Classes Details:

Classes, Objects

Constructors - default, inline, parameterised, copy - constructors always public except singleton class (private here).

Destructors - last function to run when object scope ends in a program. If A,B,C are made, C,B,A destructors will be called.

Setter, Getter functions - get the values and set the values, should always make.

Access Modifiers: Private, Public, Protected.

Can protect data implementing functions like: give pin and get some data.

Size of classes:

CPU Padding for allotting space for class:

char a; int b; char c -> 12 bytes.

Empty Class has size 1.

Static v/s Dynamic Memory Allocation:

new -> heap memory.

student *S = new Student(parameters);

(*S).name or S -> name: both can be used.

Four Pillars:

Abstraction

hiding the complexity of class methods and letting user only see the methods they can call.

Encapsulation

lets us write structured code while controlled access using access modifiers.
provides security layer.

Access Modifiers

1. Way to define access to the data and methods of a class.

2. Public

1. Members declared as public are accessible from any part of the program.
2. They can be accessed by objects of the class and external code.

3. Private

1. Members declared as private are accessible only within the same class.
2. They are not accessible from outside the class, including derived classes.

4. Protected

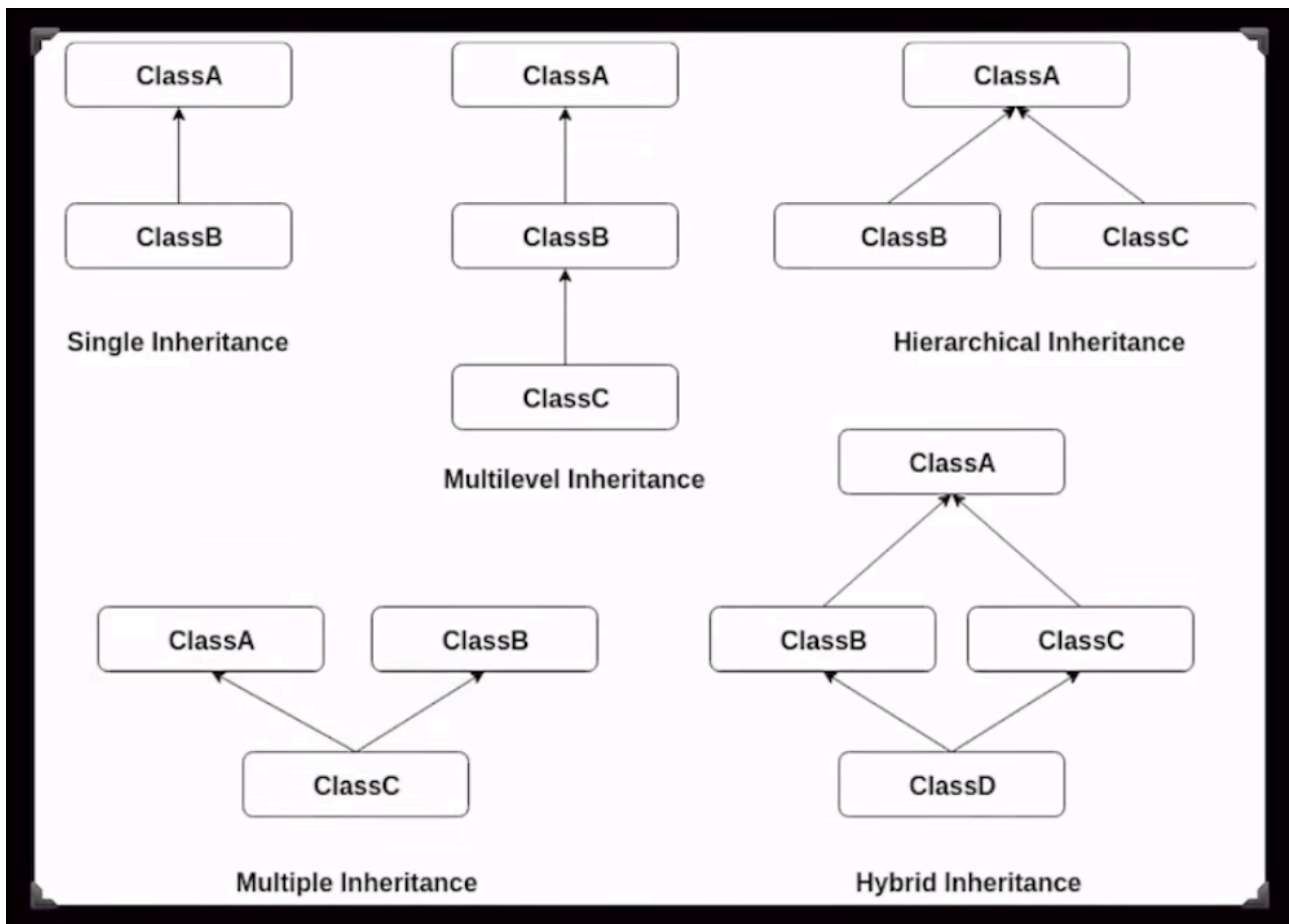
1. Members declared as protected are accessible within the same class and by derived classes.
2. They are not directly accessible from external code.

Inheritance

lets us follow the DRY principle by allowing to get functions from parent classes.
denotes "is-a" relationship.

	Derived Class	Derived Class	Derived Class
Base Class	Private Mode	Protected Mode	Public Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Types:



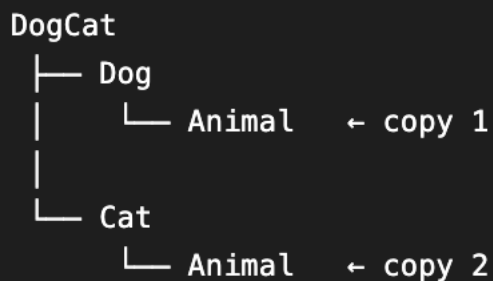
Diamond Problem during Inheritance:



DogCat inherits from **Dog** and **Cat**.

But both **Dog** and **Cat** independently inherited from **Animal**.

Therefore **DogCat** actually contains **TWO** copies of **Animal**:



Polymorphism

different forms of same function.

Types:

Compile Time - Static type, function overloading and operator overloading ("=" returns reference of object of same class).

Run Time - Dynamic type, function overriding, upcasting and down-casting.

Code:

Operator Overloading:

```

Complex operator + (const Complex &obj) {
    Complex res;
    res.real = real + obj.real;
    res.imag = imag + obj.imag;
    return res;
}
  
```

Virtual Keyword - for runtime polymorphism:

This late binds the function be called at runtime that function should be called for base class or the child class.

Up and Down Casting:

```
// virtual keyword
Shape *s = new Shape();
s->draw();

// UPCASTING
Shape *s3 = new Circle();
s3->draw();

Circle *c = new Circle();
c->draw();

// DOWNCASTING
Shape *s2 = new Shape();
Circle *c2 = (Circle *)s2;
c2->draw();
return 0;
```

Final Keyword:

Either used with class or virtual keyword.

meaning - can't override in child class when base class function is declared final.

Final Keyword

1. In C++, the final specifier is used in two main contexts: with classes and with virtual member functions.
2. **Prevents Class Inheritance:** When you declare a class as final, it means that no other class can inherit from it.
3. **Preventing Virtual Function Overriding:** The final specifier can also be used with virtual functions to prevent them from being overridden in derived classes.

Abstraction:

1. Delivering only essential information to the outer world while masking the background details.
2. It is a design and programming method that separates the interface from the implementation.

Abstract class - a class having at least 1 pure virtual function, this class can't be instantiated (no objects).

Bonus Topics:

Global and Local Variables:

most recent local variables are highest priority.

20 can't be accessed in 50's block until 50 is not removed.

global can be accessed anywhere using ::

```

int x = 2; // GLOBAL VARIABLE

int main()
{
    ::x = 4;    // global x
    int x = 20; // local to main() fn.
    cout << x << endl;
    cout << ::x << endl; // accessing Global with ::

    {
        int x = 50;
        cout << x << endl;
        cout << ::x << endl;
    }

    return 0;
}

```

Types of values in cpp:

l-value -> who have memory location.

r-value -> who does not have memory location.

Const keyword:

class members can't change in a function if function is const.

int const * a means pointer can be changes but data can't

int * const a means data can change but pointer can't be reinitialised.


```

// 2. const with pointers
// // const int *a = new int(2); // CONST data, NON-CONST poin
// int const *a = new int(2); // same as line no. 13.
// cout << *a << endl;
// // *a = 20; // cant change the content of pointer.
// // cout << *a << endl;
// int b = 20;
// a = &b; // pointer itself can be reassigned.
// cout << *a << endl;

// CONST Pointer, but NON-CONST data.
// int *const a = new int(2);
// cout << *a << endl;
// *a = 20; // chal jayega
// cout << *a << endl;
// int b = 50;
// a = &b; // nahi chalega, CONST pointer.

// CONST pointer, CONST data
// const int *const a = new int(10);
// cout << *a << endl;
// *a = 50;
// int b = 100;
// a = &b;
return 0;

```

Initialisation List:

Just a way to write constructor.

```

// ctor : old style
// abc(int _x, int _y, int _z = 0)
// {
//     x = _x;
//     y = new int(_y);
//     z = _z;
// }

// initialization list
abc(int _x, int _y, int _z = 0) : x(_x), y(new int(_y)), z(_z) {}

```

MACROS Keyword:

These do not take up any space as they are just replaced in preprocessing step of coding being converted to machine code.

```
#define PI 3.14  
#define MAXX(x, y) (x > y ? x : y)
```

Static Keyword in classes:

All the objects of a class will share the common static variables and in a static function we can ONLY access static variables because in static functions there is no instance passed in these functions.

Shallow And Deep Copy:

shallow - when we copy one object to another and heap pointers point to same memory location.

deep - each variable gets its own memory and new pointers.

Copy constructor makes shallow copy. So we have to write copy constructors ourselves to make good deep copies.

Friend Keyword in C++:

Used to access private members of any class in some other class. We can make a function friend.


```

class A
{
private:
    int x;

public:
    A(int _val) : x(_val){};

    int getX() const { return x; }
    void setX(int _val) { x = _val; }

    friend class B;
};

class B
{
public:
    void print(const A &a)
    {
        // cout << a.getX() << endl;
        cout << a.x << endl;
    }
};

int main()
{
    A a(5);
    B b;
    b.print(a);
    return 0;
}

```

V-Tables:

A **v-table** is a compiler-generated table containing pointers to the implementations of a class's virtual functions.

These are used to implement runtime polymorphism / dynamic dispatch when a class has virtual functions.

A **v-ptr** is a hidden pointer inside an object that points to the appropriate v-table.

there can't be virtual constructor but virtual destructor is possible.

virtual constructor is not possible as there is no v-table before the constructor is called, so no mapping.

Inline Functions:

Saves the stack call when used inline. Beneficial only when functions are small, else increases file size.

1. An inline function is a regular function that is defined by the inline keyword.
2. The code for an inline function is inserted directly into the code of the calling function by compiler while compiling, which can result in faster execution and less overhead compared to regular function calls.
3. Instead of calling function the statements of functions are pasted in calling function.
4. Used with small sized functions. So that executables are small (handled automatically by compiler optimisation levels).